ChatGPT

JavaScript Info Tracker

Topics Covered

- Variables and Data Types
- Scope and Closures
- Functions
- ES6+ Features
- Promises and Async/Await
- Event Loop
- DOM Manipulation
- Modules and Imports
- Error Handling
- Best Practices

Variables and Data Types

- `var`, `let`, `const` — `var` is function-scoped and can be redeclared, while `let` and `const` are block-scoped. Use `const` for values that shouldn't change, and `let` for values that may be reassigned.
- Primitive types: string, number, boolean, null, undefined, symbol, bigint
- Reference types: Object, Array, Function

Functions

- Function declarations vs expressions
- Arrow functions
- Callback functions
- Higher-order functions

Scope and Closures

- Global vs local scope
- Lexical scoping
- Closures and their use cases

Promises and Async/Await

- Creating and using Promises — Promises provide a cleaner alternative to deeply nested callbacks, often referred to as 'callback hell.' For example:

```
// Using callbacks
doSomething(function(result) {
```

```
  doSomethingElse(result, function(newResult) {
    doThirdThing(newResult, function(finalResult) {
      console.log('Got the final result: ' + finalResult);
    });
  });
});

// Using Promises
doSomething()
  .then(result => doSomethingElse(result))
  .then(newResult => doThirdThing(newResult))
  .then(finalResult => console.log('Got the final result: ' +
finalResult));
```

- Chaining `.then()` and `.catch()`
- `async` / `await` syntax
- Error handling with `try` / `catch`

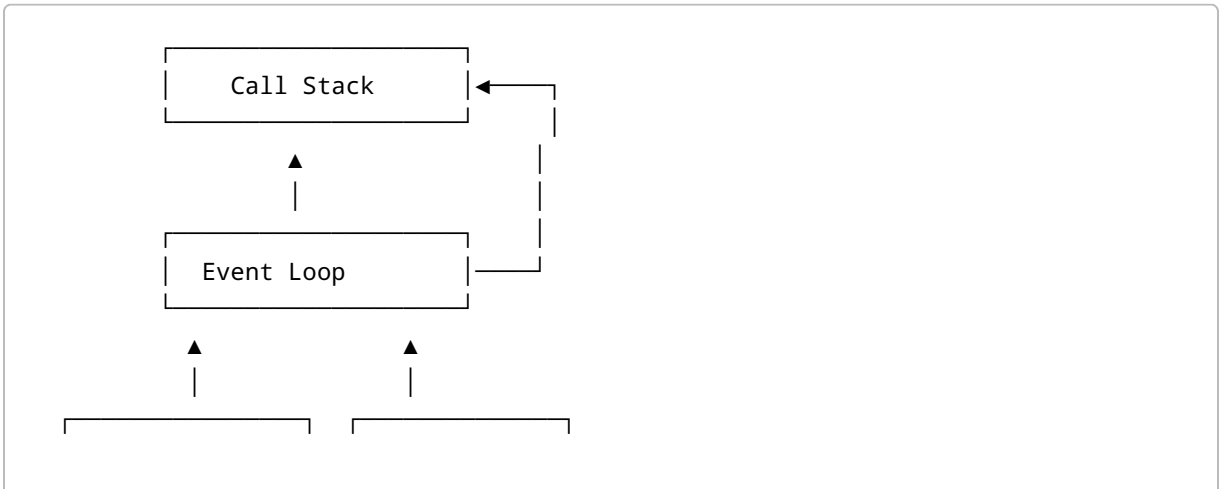
Event Loop

- Call stack
- Task queue (macrotasks) and microtasks
- How the event loop prioritizes tasks

Visual Explanation:

Here's a simplified flow of how the event loop works:

1. **Call Stack** — Executes function calls.
2. **Web APIs / Background Tasks** — Handles async operations (like setTimeout, fetch).
3. **Callback Queue (Task Queue)** — Queued callbacks from Web APIs.
4. **Microtask Queue** — Contains `.then()` handlers and `async/await` resolutions.
5. **Event Loop** — Checks if the call stack is empty, then pushes tasks from queues.



```
  | Callback Queue |   | Microtask Q  |
  └────────────────┘   └──────────────┘
```

DOM Manipulation

- Selecting elements: `getElementById`, `querySelector`
- Modifying elements: `innerText`, `innerHTML`, `setAttribute`
- Event listeners: `addEventListener`

ES6+ Features

- Template literals
- Destructuring
- Spread/rest operators
- Default parameters
- Classes
- Optional chaining and nullish coalescing

Modules and Imports

- `export` and `import`
- Named vs default exports
- Dynamic imports

Error Handling

- `try`, `catch`, `finally`
- Throwing errors
- Custom error classes

Best Practices

- Use `const`/`let` instead of `var` — `var` is function-scoped, can be redeclared, and is hoisted in a way that can lead to unexpected behavior. `let` and `const` are block-scoped and help avoid these issues.
- Keep functions pure when possible
- Avoid deep nesting
- Use meaningful names
- Linting and formatting with tools like ESLint and Prettier

Notes

(Add your custom notes, examples, or code snippets below)